

ECOM EXPERTS

Ecom productmonitor

Manual de usuario, técnico y de desarrollo

ecom_productmonitor 1.2.0 · 07/09/2026

Contenido

¿Qué hace este módulo?

Caso: "mis campos obligatorios de dirección se desactivan solos"

Instalación

Cómo se usa

Ver el histórico

Productos y combinaciones

Cambios de configuración de módulos

Limpiar todo el historial

Ver el detalle de un cambio

Orígenes que detecta

Configuración

Multitienda

Preguntas frecuentes

1. Propósito

2. Arquitectura (legacy, máxima compatibilidad)

3. Hooks registrados

3.bis Vigilancia de campos obligatorios y configuración

Cambios de configuración de MÓDULOS (ECOM_PM_ENABLE_MODULES, entity_type module)

SEGURIDAD: enmascarado de secretos (obligatorio)

installDefaults() (bug de actualización)

Autodiagnóstico (runDiagnostics())

Despliegue de overrides (manual y reversible)

4. Detección de origen (EcomProductMonitorDetector::detect)

5. Diff por snapshot (EcomProductMonitorRecorder)

6. Base de datos

7. Multitienda

8. Seguridad y rendimiento

Claves de configuración

9. Mantenimiento

1. Flujo de captura

2. Extender el monitor a más campos o entidades

Añadir etiqueta a un campo nuevo

Ignorar un campo ruidoso

Monitorizar otra entidad (p. ej. Fabricante)

Combinaciones (Combination)

Cambios de configuración de módulos (entity_type='module')

Limpiar historial

2.bis Vigilancia de required fields y configuración

Overrides (despliegue manual, regla del proyecto)

3. Detección de origen — añadir un canal

4. Base de datos

5. Grid (AdminEcomProductMonitorController)

6. Convenciones aplicadas

7. Pruebas recomendadas

¿Qué hace este módulo?

Vigila automáticamente **todos los cambios** que se hacen en tus **productos, categorías, stock, campos obligatorios de dirección y configuración de la tienda**, y te dice, por cada cambio: **qué** se cambió (campo por campo, valor antes → valor después), **quién** lo hizo y **cómo/desde dónde**, con **fecha y hora**. Guarda un **histórico** completo y consultable.

Caso: "mis campos obligatorios de dirección se desactivan solos"

Si activas campos requeridos (Clientes > Direcciones > *Definir campos obligatorios*) y al cabo de unas horas o al día siguiente vuelven a estar desactivados, este módulo te dirá **quién o qué los cambió y desde dónde** (un módulo, una tarea programada/CRON nocturna, un empleado, una importación...).

Cómo activarlo: 1. En *Configurar*, deja **Monitorizar campos obligatorios de dirección** en **Sí** (viene activado). 2. Para saber el origen EXACTO (no solo cuándo), pulsa **Desplegar overrides** en el panel *Captura forense avanzada*. Es seguro y reversible (botón **Retirar overrides**). 3. Vuelve a fijar tus campos obligatorios y espera a que se reproduzca el reset. En **Monitor de cambios**, filtra por tipo **Campos obligatorios**: verás la fila con el origen (módulo/CRON/empleado), la fecha exacta y qué campos se desactivaron.

Sin overrides el módulo también lo detecta, pero el "quién" mostrado será la persona que entró al back office y disparó la detección, no necesariamente quien lo cambió. Con overrides, el origen es exacto.

Instalación

1. Sube la carpeta `ecom_productmonitor` a `/modules/` (o el ZIP desde *Módulos > Subir un módulo*).
2. Pulsa **Instalar**.
3. Aparecerá un nuevo menú: **Clientes > Monitor de cambios** (o en *Configurar*, según tu versión).

Cómo se usa

Ver el histórico

Entra en **Monitor de cambios**. Verás una tabla con una fila por cada cambio: - **ID** del registro y **Fecha** (con hora). - **Tipo**: Producto o Categoría. - **ID producto/cat.** y **Referencia y Nombre**. - **Acción**: Alta, Modificación, Eliminación o Stock. - **Razón del cambio**: resumen de qué campos cambiaron. - **Origen (quién)**: empleado, API, importación CSV, módulo, CRON...

Puedes **filtrar** por tipo, acción, referencia, nombre y origen, y **ordenar** por fecha. También **exportar** a CSV.

Productos y combinaciones

El módulo distingue cuándo se cambia **el producto** y cuándo se cambia **una combinación** (talla, color...) de ese producto: - Filtra por tipo **Producto** para ver cambios del producto en sí. - Filtra por tipo **Combinación** para ver altas/cambios/bajas de combinaciones. El nombre muestra el producto y los atributos (p. ej. *Camiseta - M, Rojo*) y la razón incluye *[Producto #123]* para saber a qué producto pertenece.

Cambios de configuración de módulos

Con el interruptor **Monitorizar configuración de módulos** en **Sí** (viene activado), el monitor registra **todos** los cambios de configuración de **cualquier módulo** de tu tienda: cada vez que alguien entra en *Configurar* de un módulo y guarda, verás una fila por cada ajuste cambiado, con el **valor anterior y el nuevo**, **quién** lo hizo y **desde dónde**.

Filtra por tipo **Módulo** en el listado. La columna *Nombre* muestra el módulo afectado y la *Razón* indica el ajuste concreto (p. ej. *Config del modulo "ps_checkout" modificada: PS_CHECKOUT_MODE*).

Requiere haber pulsado **Desplegar overrides** (panel *Captura forense avanzada*), porque la configuración de los módulos no dispara ningún aviso nativo de PrestaShop: hay que interceptarla.

Tus secretos están protegidos: si el ajuste que cambia es una contraseña, una clave de API o un token (de una pasarela de pago, un transportista, etc.), el monitor registra **que cambió, quién y cuándo**, pero **nunca guarda el valor**: verás (*valor oculto*). Así el histórico —que es visible en el back office y exportable a CSV— no filtra credenciales.

Limpiar todo el historial

En el listado, arriba, tienes el botón rojo **Limpiar TODO el historial**. Pide confirmación y borra **todos** los registros de cambios de forma permanente (no se puede deshacer). No afecta a la detección futura: a partir de ahí se siguen registrando los nuevos cambios con normalidad.

Ver el detalle de un cambio

Haz clic en **Ver** (lupa) en cualquier fila. Se abre el detalle completo: - **Qué se ha cambiado**: tipo, acción, ID, referencia, nombre, razón, fecha/hora, tienda. - **Quién y cómo**: origen, empleado (si aplica), módulo responsable, huella de la clave API (si fue por API), controlador, método HTTP, URL/ruta, IP y navegador. - **Campos modificados (antes / después)**: tabla línea a línea con el campo, su código técnico, el valor anterior (rojo, tachado) y el nuevo (verde). Incluye cambios por idioma (p. ej. *Nombre [ES], Nombre [EN]*).

Orígenes que detecta

Origen	Significado
Edición manual back office	Un empleado lo cambió desde el panel
API Webservice (sistema externo)	Una integración/ERP/app externa vía API
Importación CSV	Una importación desde <i>Parámetros avanzados > Importar</i>
Módulo "x"	Otro módulo realizó el cambio
CRON / CLI	Una tarea programada o por consola
Front office (cliente)	Acción originada en la tienda
Origen externo / script	No identificado

Configuración

En *Módulos > Ecom Product Monitor > Configurar*: - **Monitorizar productos / categorías / combinaciones / stock**: activa o desactiva cada tipo. - **Monitorizar campos obligatorios de dirección**: vigila quién activa/desactiva los campos requeridos (recomendado: Sí). - **Monitorizar configuración**: registra cambios en la configuración de la tienda (requiere desplegar los overrides; puede generar muchos registros, usa la whitelist). - **Whitelist de claves de configuración**: si la rellenas, solo se vigilan esas claves (una por línea o separadas por comas). Si la dejas vacía, se vigilan todas menos el ruido habitual. - **Días de retención**: cuántos días se guardan los registros (por defecto 90). Los más antiguos se borran solos. - **Modo depuración**: guarda logs técnicos en `/logs` (solo para soporte). - **Captura forense avanzada (overrides)**: botones para **Desplegar** / **Retirar** los overlays que capturan el origen exacto de los cambios de campos obligatorios y configuración.

Multitienda

Compatible. Cada cambio queda asociado a su tienda y, si trabajas con una tienda concreta seleccionada, solo verás los cambios de esa tienda.

Preguntas frecuentes

¿Ralentiza la tienda? No: solo registra cuando ocurre un cambio y nunca bloquea el guardado. **¿Envía datos a terceros?** No registra acciones externas; solo guarda el histórico internamente (salvo el registro de licencia del módulo). **La primera vez que edito un producto antiguo dice "sin línea base previa"**. Es normal: a partir de ese momento ya queda registrada la línea base y los siguientes cambios mostrarán el detalle completo.

Ecom Product Monitor — Documentación Técnica

Versión: 1.1.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0 · **Compatibilidad:** PrestaShop 1.7 / 8 / 9 · PHP 7.4–8.x · Multitienda.

1. Propósito

Monitor forense de cambios sobre **productos, categorías, stock, campos obligatorios de dirección** (`ps_required_field`) y **configuración** (`ps_configuration`). Por cada cambio registra **qué** cambió (diff campo a campo antes/después), **quién** lo hizo y **cómo/desde dónde** (canal de entrada + traza). No ejecuta acciones hacia terceros: solo registra de forma reactiva mediante hooks y, opcionalmente, overrides forenses.

2. Arquitectura (legacy, máxima compatibilidad)

```
ecom_productmonitor/
├─ ecom_productmonitor.php           Clase principal Ecom_Productmonitor
(install/hooks/config)
├─ classes/
│   └─ EcomProductMonitorLog.php      ObjectModel del registro (binding del grid + vista)
│   └─ EcomProductMonitorDetector.php Detección de origen (quién/cómo)
│   └─ EcomProductMonitorRecorder.php Diff por snapshot + persistencia del log (insertLog público)
│   └─ EcomProductMonitorConfigRecorder.php Required fields + ps_configuration (reutiliza Detector/insertLog)
├─ EcomProductMonitorLogger.php       Logger de depuración a /logs (toggle)
├─ override_v8/classes/               Overrides forenses (despliegue MANUAL, nunca en install)
│   └─ ObjectModel.php                Intercepta addFieldsRequiredDatabase()
│   └─ Configuration.php              Intercepta updateValue()/deleteByName()
├─ controllers/admin/
│   └─ AdminEcomProductMonitorController.php Grid (HelperList) + renderView de detalle
├─ views/templates/admin/             detail.tpl, configure.tpl, gmartos_update.tpl
├─ views/css/monitor.css
└─ logs/                             Logs de depuración (si ECOM_PM_DEBUG=1)
```

Se eligió arquitectura legacy (`ModuleAdminController` + `HelperList`) por ser la más robusta y portable entre 1.7/8/9, evitando dependencias de Symfony grids/Doctrine/services.yml. Sin Composer, sin namespace en la clase principal, carga manual con `require_once` .

3. Hooks registrados

Hook	Entidad	Uso
<code>actionObjectProductAddAfter</code>	Producto	Alta
<code>actionObjectProductUpdateAfter</code>	Producto	Modificación (diff)
<code>actionObjectProductDeleteAfter</code>	Producto	Eliminación
<code>actionObjectCategoryAddAfter</code>	Categoría	Alta
<code>actionObjectCategoryUpdateAfter</code>	Categoría	Modificación (diff)
<code>actionObjectCategoryDeleteAfter</code>	Categoría	Eliminación
<code>actionObjectCombinationAddAfter</code>	Combinación	Alta (enlazada a su producto)
<code>actionObjectCombinationUpdateAfter</code>	Combinación	Modificación (diff: atributos, precio impacto, referencia...)
<code>actionObjectCombinationDeleteAfter</code>	Combinación	Eliminación
<code>actionUpdateQuantity</code>	Stock	Cambio de stock (producto y combinación)
<code>displayBackOfficeHeader</code>	Campos obligatorios	Red de seguridad: detecta cambios en <code>ps_required_field</code> al cargar cualquier página del BO

Las combinaciones (`Combination`) se registran como `entity_type='combination' : id_entity = id de la combinación, entity_name = "Producto - Atributo1, Atributo2"`, y la razón incluye `[Producto #id_product]` para enlazar con el producto padre. El snapshot/diff es independiente del producto (clave `entity_type+id_entity+id_shop`), por lo que editar el producto y editar una de sus combinaciones generan registros separados y distinguibles. Gobernado por el toggle `ECOM_PM_ENABLE_COMBINATIONS`.

Limpieza de historial: `AdminEcomProductMonitorController::postProcess()` atiende `submitClearAllEcomPm` (botón rojo "Limpiar TODO el historial" en `renderList()`, con confirmación JS, token CSRF y permiso `delete`) → `DELETE FROM ps_ecom_productmonitor_log`. No toca la tabla `snapshot` (las líneas base se conservan para que el diff siga funcionando).

Son hooks de núcleo disparados por `ObjectModel::add/update/delete`, por lo que capturan cualquier origen (BO, API, CSV, módulos, CRON).

3.bis Vigilancia de campos obligatorios y configuración

El caso de uso: en algunas tiendas los **campos obligatorios de dirección se desactivan solos** (a veces al día siguiente). Estos campos NO están en `ps_configuration` sino en `ps_required_field` (`object_name = 'CustomerAddress'`), y se reescriben **siempre** por `ObjectModel::addFieldsRequiredDatabase($fields)`

(DELETE total por `object_name` + reinsert). No existe hook nativo para este evento, por lo que la captura precisa requiere override.

Sistema de **dos capas**: 1. **Red de seguridad (sin override, siempre activa)**: `hookDisplayBackOfficeHeader` → `EcomProductMonitorConfigRecorder::detectRequiredFieldChange()` compara el estado actual de `ps_required_field` (snapshot canónico `object_name|id_shop|field_name`) con la línea base guardada en la tabla `snapshot` (`entity_type='required_field'`). Si difiere, registra el cambio. Captura correctamente las ediciones manuales del BO (corre tras `postProcess`); para cambios fuera de banda (CRON nocturno, INSERT directo) registra el **momento de detección** y marca explícitamente que el origen es la petición que lo detectó. 2. **Captura precisa (override opcional, `ECOM_PM_ENABLE_REQUIRED_FIELDS`)**: `override_v8/classes/ObjectModel.php` envuelve `addFieldsRequiredDatabase()`: snapshot antes → `parent::` → snapshot después → `recordRequiredFields(..., 'live')` con la **traza exacta** (`debug_backtrace` → módulo culpable, controlador, CLI/CRON, empleado, IP). Tras registrar, actualiza la línea base para que la red de seguridad no duplique.

`override_v8/classes/Configuration.php` envuelve `updateValue()` / `deleteByName()`: filtro de cadena barato primero (descarta `ECOM_PM_*`, `GMARTOS_*`, `PS_LAST_*`, `PS_CCC*`, `CONF_*` → evita ruido y bucles); si al menos una de las dos vigilancias está activa, captura `old` (antes) vs `new` (después) y delega en `recordConfig()`.

Cambios de configuración de MÓDULOS (`ECOM_PM_ENABLE_MODULES`, `entity_type module`)

Toda la configuración de cualquier módulo de PrestaShop se guarda vía `Configuration::updateValue()`, así que el override ya la intercepta. Lo que hace falta es **atribuir la clave al módulo** que la escribe.

`EcomProductMonitorConfigRecorder::resolveModuleForConfig()` usa dos señales, de más fiable a menos: 1. **Tools::getValue('configure')** — estamos en la página *Configurar* de un módulo (`AdminModules&configure=<nombre>`); se valida con `Validate::isModuleName()`. Cubre el caso normal de configurar/modificar un módulo desde el BO. 2. **Backtrace** (`EcomProductMonitorDetector::detectCallingModule()`, ahora público) — el primer frame bajo `/modules/<nombre>/`. Cubre módulos con controlador propio, hooks y CRON.

Si la clave es atribuible a un módulo → se registra como `entity_type='module'` (`entity_name` = nombre del módulo, `id_entity` = `id_module`, `reference` = nombre), gobernado por `ECOM_PM_ENABLE_MODULES` e **ignorando la whitelist** (esa solo aplica a la config genérica de PrestaShop). Si no es atribuible → comportamiento anterior: `entity_type='config'`, gobernado por `ECOM_PM_ENABLE_CONFIG` + `ECOM_PM_CONFIG_WHITELIST`.

`entity_type` es `VARCHAR(16)`: `module` (6), `config` (6), `combination` (11), `required_field` (14) — todos caben.

Tercera señal de atribución: si el módulo usa su propio `ModuleAdminController` con `$fields_options`, el guardado lo ejecuta `AdminController` (del core) y el backtrace NO ve `/modules/`. En ese caso se usa `$context->controller->module->name`.

SEGURIDAD: enmascarado de secretos (obligatorio)

Registrar la configuración de **todos** los módulos implica que pasarían por el log **claves de API, contraseñas, tokens y secretos** de pasarelas de pago, transportistas, ERPs... `isSecretKey()` detecta la clave por patrón (`PASSW|PWD|SECRET|TOKEN|API_?KEY|APIKEY|PRIVATE|SALT|CREDENTIAL|CLIENT_?SECRET|AUTH|SIGNATURE|CERT`, case-insensitive) y `displayValue()` sustituye el valor por `(valor oculto)` / `(valor oculto: cambiado)`. **Se**

registra QUÉ clave cambió, quién y cuándo — nunca su valor. Esto es imprescindible porque el listado del monitor es visible en el BO y **exportable a CSV**.

`logs/.htaccess` (Deny from all, con soporte `mod_authz_core` y legacy) impide descargar por HTTP los `.log` de depuración.

`installDefaults()` (bug de actualización)

`install()` no se re-ejecuta al actualizar un módulo ya instalado, por lo que las claves NUEVAS no existían y los switches salían en **No**. `addnewfeatures()` ahora llama a `installDefaults()`, que inicializa solo las claves inexistentes (`Configuration::get()` devuelve `false` únicamente si la clave no existe) y **respeta cualquier valor ya guardado por el usuario**.

Autodiagnóstico (`runDiagnostics()`)

Panel en la pantalla *Configurar* con semáforo por comprobación: fichero de override copiado (estado **por fichero**: `deployed` / `stale` / `foreign` / `missing`), **override realmente ACTIVO en memoria**, `PS_DISABLE_OVERRIDES`, toggle de módulos, hooks registrados y filas por `entity_type`.

La prueba definitiva es el **método centinela**: cada override declara `ecomPmConfigurationOverrideActive()` / `ecomPmObjectModelOverrideActive()`, y el panel comprueba `method_exists('Configuration', ...)`. Que el fichero esté copiado **no** significa que PrestaShop lo esté cargando (caché de `class_index.php`, opcode, `PS_DISABLE_OVERRIDES` en PS ≤ 8.0); `method_exists` sí lo prueba.

Bug corregido en 1.2.0: `overridesDeployed()` usaba **OR** sobre `OVERRIDE_FILES` y devolvía `true` en cuanto encontraba **uno** de los dos ficheros (y `ObjectModel.php` se evaluaba primero). Si `Configuration.php` no llegaba a copiarse (override ajeno preexistente o permisos), la UI pintaba "desplegados y activos" en verde **y ocultaba el botón de desplegar** (estaba en el `{else}`), dejando al usuario sin salida y sin captura de config de módulos. Ahora `overrideStatus()` evalúa **fichero a fichero** y `overridesDeployed()` exige que **todos** estén `deployed`; el botón está **siempre** visible.

Nota sobre `PS_DISABLE_OVERRIDES`: desactiva los overrides de clase **solo en PrestaShop ≤ 8.0** (autoloader legacy). En 8.1+/9.x ya no los desactiva. La constante `_PS_DISABLE_OVERRIDES` **no existe** en el core: no programar contra ella.

Despliegue de overrides (manual y reversible)

Cumpliendo el estándar del proyecto (overrides V8 manuales, **nunca** `installOverrides()` ni copia en `install()`): los ficheros viven en `override_v8/classes/` y se copian a `/override/classes/` con los botones de la pantalla de configuración (`deployOverrides()` / `removeOverrides()`). El despliegue es idempotente (md5), **aborta si existe un override ajeno** (sin marcador `ECOM_PRODUCTMONITOR_OVERRIDE`) pidiendo fusión manual, e invalida `class_index.php` (`var/cache/*/` en PS8/9 y `_PS_CACHE_DIR` en versiones previas). `uninstall()` retira solo los overrides propios. Todos los overrides envuelven la lógica del módulo en `try/catch (\Throwable)` con guardas `is_file()`: si el módulo no está o falla, el core nunca se rompe.

4. Detección de origen (`EcomProductMonitorDetector::detect`)

Prioridad de resolución: 1. **CLI / CRON** → `PHP_SAPI === 'cli'` . 2. **API Webservice** → URI `/api/` , parámetro `ws_key` , cabecera `Authorization Basic` , o controlador `WebserviceRequest` . Se guarda **huella enmascarada** de la key (`abcd...wxyz`) para identificar qué integración externa. 3. **Importación CSV** → controlador `AdminImportController` . 4. **Back office (empleado)** → `_PS_ADMIN_DIR_ + context->employee` . Si además hay un módulo en la pila → `module_bo` . 5. **Módulo en front** → un frame de la pila apunta a `/modules/<otro>/` → `module_fo` . 6. **Front office (cliente)** → `context->customer` . 7. **Desconocido** → fallback.

El módulo responsable se detecta con `debug_backtrace()` buscando el primer frame bajo `/modules/` distinto a `ecom_productmonitor` .

Metadatos forenses capturados: controlador, método HTTP, URI/ruta, IP (`Tools::getRemoteAddr`), user-agent, empleado (id+nombre), módulo responsable, huella de key API.

5. Diff por snapshot (`EcomProductMonitorRecorder`)

- Tabla `snapshot` guarda la **línea base** (JSON) de los campos monitorizados por entidad **y tienda**.
- En `add` → crea snapshot. En `update` → compara snapshot anterior vs estado actual, registra solo los campos que cambiaron y refresca el snapshot. En `delete` → elimina snapshot.
- Si no existe línea base previa (entidad antigua nunca tocada): se registra la modificación con razón *"Cambio detectado (sin línea base previa)"* y se crea el snapshot. **No hay seed masivo**: la línea base se crea de forma perezosa al primer cambio real.
- **Cobertura total**: el snapshot se construye dinámicamente desde `ObjectModel::getDefinition()` , capturando **todos** los campos del producto/categoría (todos los códigos: `reference` , `ean13` , `upc` , `isbn` , `mpn` , `supplier_reference` , `location` ; precios, impuestos, dimensiones, visibilidad, redirecciones...) y **todos los campos multilingües en todos los idiomas** (clave `campo|idLang|iso`). Se excluyen solo campos de ruido (`date_add` , `date_upd` , `indexed` , `cache_*`).
- Cada línea del diff guarda `field` (etiqueta localizada, con sufijo de idioma `[ES]`), `code` (nombre técnico del campo), `old` y `new` . Valores numéricos normalizados para evitar falsos positivos de formato.

6. Base de datos

- `ps_ecom_productmonitor_log` : registro de cambios (incluye `id_shop` , `id_shop_group` , metadatos forenses y `changes` JSON). Índices por entidad, tienda y fecha.
- `ps_ecom_productmonitor_snapshot` : línea base por entidad/tienda (`UNIQUE(entity_type,id_entity,id_shop)`).
- Charset `utf8mb4` , motor `_MYSQL_ENGINE_` . SQL defensivo (`IF NOT EXISTS` , `DROP ... IF EXISTS` , pSQL).

7. Multitienda

- `id_shop` / `id_shop_group` en cada registro y snapshot.
- Listado restringido por `Shop::getContextListShopID()` salvo contexto "Todas las tiendas".

- Snapshots independientes por tienda → el diff es correcto cuando un producto tiene valores distintos por tienda.

8. Seguridad y rendimiento

- `if (!defined('_PS_VERSION_')) exit;` en todos los PHP; `index.php` recursivos.
- Salida escapada en Smarty (`escape:'html':'UTF-8'`), sin `nofilter`.
- Captura envuelta en try/catch: un fallo del monitor nunca rompe el guardado del producto.
- Purga interna throttled (1/día) según `ECOM_PM_RETENTION_DAYS` (default 90). Solo `DELETE` interno, sin acciones externas (cumple Regla 0).
- Registro en `modules.gmartos.es` (ping semanal, timeout 3s, fallo en silencio) mediante **cURL nativo** con fallback a `file_get_contents` (sin dependencias de Composer; se eliminó el uso de `GuzzleHttp\Client` que podía provocar *fatal* si no estaba autoload). El ping solo se marca tras respuesta válida → si falla, se reintenta en la siguiente visita.
- **Overrides defensivos:** filtro de cadena antes de cualquier consulta (rendimiento en `Configuration::updateValue`, que es caliente); vigilancia de configuración **desactivada por defecto** (`ECOM_PM_ENABLE_CONFIG=0`).

Claves de configuración

Clave	Default	Uso
<code>ECOM_PM_ENABLE_PRODUCTS</code> / <code>_CATEGORIES</code> / <code>_STOCK</code>	1	Toggles de monitorización existentes
<code>ECOM_PM_ENABLE_REQUIRED_FIELDS</code>	1	Campos obligatorios de dirección
<code>ECOM_PM_ENABLE_CONFIG</code>	0	Cambios en <code>ps_configuration</code> (requiere override)
<code>ECOM_PM_CONFIG_WHITELIST</code>	"	Claves a vigilar (vacío = todas menos ruido)
<code>ECOM_PM_RETENTION_DAYS</code>	90	Retención de registros
<code>ECOM_PM_DEBUG</code>	0	Logs técnicos en <code>/logs</code>

9. Mantenimiento

`addnewfeatures()` se ejecuta al entrar en configuración: reinstala tablas/hooks/tab si faltan (idempotente). No se usan ficheros `upgrade`.

Ecom Product Monitor — Guía de Desarrollo

Versión: 1.1.0 · **Autor:** Ecom Experts · **Licencia:** AFL-3.0

1. Flujo de captura

```
ObjectModel::update($product)
├─ Hook::exec('actionObjectProductUpdateAfter', ['object' => $product])
├─ Ecom_Productmonitor::hookActionObjectProductUpdateAfter()
├─ capture('product', 'update', $object)
├─ EcomProductMonitorRecorder::record()
│   ├── snapshotData() → estado actual (todos los campos/idiomas)
│   ├── loadSnapshot() → línea base anterior
│   ├── diff() → cambios línea a línea
│   ├── saveSnapshot() → actualiza línea base
│   └─ insertLog()
│       └─ EcomProductMonitorDetector::detect() → quién/cómo
```

Toda la captura está dentro de `try/catch`: un error del monitor **nunca** interrumpe el guardado de la entidad.

2. Extender el monitor a más campos o entidades

Añadir etiqueta a un campo nuevo

La captura ya es automática (lee la definición del ObjectModel). Solo hay que dar nombre legible en `EcomProductMonitorRecorder::fieldLabels()`:

```
'mi_campo' => 'Mi etiqueta',
```

Si no se añade, el diff usará el nombre técnico del campo.

Ignorar un campo ruidoso

Añádelo a `EcomProductMonitorRecorder::ignoredFields()`.

Monitorizar otra entidad (p. ej. Fabricante)

1. Registrar los hooks en `registerHooks()`: `actionObjectManufacturerAddAfter/UpdateAfter/DeleteAfter`.
2. Declarar los métodos `hookActionObjectManufacturer*` que llamen a `capture('manufacturer', ...)`.
3. `record()` y `snapshotData()` ya son genéricos (leen la definición del ObjectModel), no requieren cambios.

Combinaciones (Combination)

Implementadas como entidad `combination` con los hooks

`actionObjectCombinationAddAfter/UpdateAfter/DeleteAfter` (toggle `ECOM_PM_ENABLE_COMBINATIONS`).

`record()` se generalizó: para `entity_type='combination'` el `entity_name` se resuelve con `comboName()`

(producto padre vía `new Product($id_product)` + atributos vía join `product_attribute_combination` ×

`attribute_lang`) y la razón añade `[Producto #id_product]`. El diff usa snapshot propio

(`entity_type+id_entity+id_shop`), así que un cambio en el producto y un cambio en su combinación son

registros separados. El stock de combinación ya lo cubría `actionUpdateQuantity` (anota (*combinacion* #X)).

Cambios de configuración de módulos (entity_type='module')

No existe hook nativo para "un módulo ha guardado su configuración". El punto de intercepción es

`Configuration::updateValue()` / `deleteByName()` (override ya existente), porque **toda** la config de módulos pasa por ahí.

`EcomProductMonitorConfigRecorder::resolveModuleForConfig($ctx)` atribuye la clave a un módulo: 1.

`Tools::getValue('configure')` + `Validate::isModuleName()` → página *Configurar* del módulo (señal fuerte, cubre el flujo normal del BO). 2. `EcomProductMonitorDetector::detectCallingModule()` (hecho **público**) → primer frame del backtrace bajo `/modules/<name>/`; cubre controladores propios del módulo, hooks y CRON.

3. Devuelve `''` si no es atribuible → se trata como config genérica.

`recordConfig()` bifurca: módulo → `entity_type='module'` con `ECOM_PM_ENABLE_MODULES`, **sin whitelist**;

genérico → `entity_type='config'` con `ECOM_PM_ENABLE_CONFIG` + whitelist. El filtro anti-bucle `isNoiseKey()` (prefijos `ECOM_PM_`, `GMARTOS_`, `PS_LAST_`, `PS_CCC`, `CONF_`) se aplica **siempre**, en el override y en el recorder.

Rendimiento: el override filtra por prefijo (solo strings) antes de tocar la BD, y aborta si ambos toggles están

OFF. `resolveModuleForConfig()` prueba primero `configure` (barato) y solo cae al `debug_backtrace` si hace falta.

Limpiar historial

`AdminEcomProductMonitorController::renderList()` antepone un `<form>` con botón `submitClearAllEcomPm`

(confirmación JS, action a `self::$currentIndex&token`, requiere `access('delete')`). `postProcess()` lo

atiende con `DELETE FROM ps_ecom_productmonitor_log` (no `TRUNCATE`, para no exigir privilegios extra ni chocar con FKs) y deja intacta la tabla `snapshot`.

2.bis Vigilancia de required fields y configuración

`EcomProductMonitorConfigRecorder` reutiliza `EcomProductMonitorDetector::detect()` y

`EcomProductMonitorRecorder::insertLog()` (ahora **público**). Dos entidades nuevas en la tabla `log`:

`entity_type='required_field'` y `entity_type='config'`.

Flujo required fields (dos capas, sin duplicar):

```

addFieldsRequiredDatabase($fields)           ← override (captura precisa, modo 'live')
├─ snapshotRequiredFields() (antes)           estado canónico object_name|id_shop|field_name
├─ parent::addFieldsRequiredDatabase()
├─ snapshotRequiredFields() (después)
├─ recordRequiredFields(before, after, ctx, 'live')
│   ├─ diff added/removed → changes
│   ├─ insertLog() (Detector → módulo/CRON/empleador exacto)
│   └─ saveRequiredSnapshot(after)             actualiza línea base → la red de seguridad no
duplica
hookDisplayBackOfficeHeader()                 ← red de seguridad (modo 'detected', sin
override)
├─ detectRequiredFieldChange(ctx)
│   └─ si current != línea base → recordRequiredFields(prev, current, ctx, 'detected')

```

Snapshot de required fields: se guarda en la tabla `snapshot` con `entity_type='required_field'`, `id_entity=0`, `id_shop=0` (lista JSON), reutilizando el `UNIQUE` existente. No requiere cambios de esquema.

Flujo configuración: el override `Configuration::updateValue()/deleteByName()` llama a `recordConfig($key,$old,$new,$action,$ctx)`. Filtro `isWatchedConfigKey()`: descarta prefijos de ruido (`ECOM_PM_`, `GMARTOS_`, `PS_LAST_`, `PS_CCC`, `CONF_`); si `ECOM_PM_CONFIG_WHITELIST` tiene contenido, vigila solo esas claves.

Overrides (despliegue manual, regla del proyecto)

- Viven en `override_v8/classes/` con el marcador `ECOM_PRODUCTMONITOR_OVERRIDE`.
- `deployOverrides()` copia a `/override/classes/` solo si: no existe destino, o existe y es nuestro (marcador) y difiere (md5). Si existe un override **ajeno**, aborta y pide fusión manual.
- `invalidateClassIndex()` borra `var/cache/*/class_index.php` (PS8/9) y `_PS_CACHE_DIR/class_index.php`.
- `removeOverrides()` (también en `uninstall()`) borra solo ficheros con nuestro marcador.
- Firma de `addFieldsRequiredDatabase($fields)` **sin type hint** → compatible (contravarianza) con cualquier versión del core. Todo envuelto en `try/catch (\Throwable)` + `is_file()`: el core nunca se rompe.

3. Detección de origen — añadir un canal

En `EcomProductMonitorDetector::detect()` el orden de prioridad importa. Para añadir un canal nuevo (p. ej. una cola/worker propio), inserta una comprobación antes del fallback y devuelve `source_type` + `source_label` localizado. Para identificar módulos se usa `debug_backtrace()` (`detectCallingModule`).

4. Base de datos

- `ecom_productmonitor_log` — append-only. La columna `changes` es JSON: `[{field, code, old, new}, ...]`.

- `ecom_productmonitor_snapshot` — línea base por `entity_type + id_entity + id_shop` (UNIQUE). Se actualiza con `INSERT ... ON DUPLICATE KEY UPDATE`.

Reglas seguidas: cast a entero de IDs, `pSQL()` en todo string (incluido `pSQL(json, true)`), sin `LIMIT 1` manual, `IF NOT EXISTS` / `DROP IF EXISTS`.

5. Grid (AdminEcomProductMonitorController)

- `HelperList` legacy con `$this->className = 'EcomProductMonitorLog'`.
- Columnas con `callback` (`renderEntityType`, `renderAction`) y filtros `select`.
- Solo lectura: se elimina el botón "Nuevo" en `initToolbar()/initPageHeaderToolbar()`. Acción `view` → `renderView()` que pinta `detail.tpl`.
- Multitienda: `_where` restringe por `Shop::getContextListShopID()`.

6. Convenciones aplicadas

- Compatibilidad PHP 7.4–8.x: sin typed properties, sin `match`, sin promoción de constructor, sin `::class` sobre objetos (`get_class()`).
- `if (!defined('_PS_VERSION_')) exit;` en todos los PHP; `index.php` en cada carpeta.
- Sistema de traducción nuevo (`isUsingNewTranslationSystem` → true), dominio `Modules.Ecomproductmonitor.Admin`.
- Salida Smarty escapada, sin `nofilter`.
- Registro Gmartos + `addnewfeatures()` idempotente.

7. Pruebas recomendadas

1. Editar un producto en BO → registro con origen "Edición manual back office" + diff de campos.
2. Cambiar precio por API Webservice → origen "API Webservice" + huella de la key.
3. Importar un CSV de productos → origen "Importación CSV".
4. Lanzar un módulo de sincronización → origen "Módulo \"x\"".
5. Cambiar stock → acción "Stock".
6. Multitienda: editar el mismo producto en dos tiendas → dos snapshots y diffs independientes.
7. Sin overrides: cambiar campos obligatorios en *Clientes* > *Direcciones* → al recargar el BO aparece registro tipo "Campos obligatorios" con origen del empleado.
8. Con overrides desplegados: cambiar campos obligatorios → registro inmediato modo `live` con la traza exacta; lanzar por consola un script que invoque `addFieldsRequiredDatabase()` → origen "CRON / CLI" + módulo culpable por `backtrace`.
9. `ECOM_PM_ENABLE_CONFIG=1` + whitelist `PS_B2B_ENABLE` → cambiar B2B en BO → registro tipo "Configuración" con old/new.
10. Desplegar/retirar overrides desde *Configurar* → comprobar que aparecen/desaparecen en `/override/classes/` y que se invalida `class_index.php`.